

Lab 5 – Secure Development/Coding

Develop a web-based student portal for a university (Based on Lab 1 and Lab 3). The portal allows students to:

- Log in to view grades and attendance
- Submit assignments
- Communicate with lecturers via messaging
- Update personal information

Continue: Scenario

Excellent! You have successfully identified the threat and possible mitigation from manual (Lab1) and using Threat Modeling Tools (Lab 3). Next, develop the application with the proposed mitigation. (You may also use this example lab code for the exercise)

Step 1 — Create a project folder in Kali

- Create a project folder

```
mkdir -p ~/student_portal_lab_secure
```

```
cd ~/student_portal_lab_secure
```

- Move or copy the downloaded zip file into this folder.

Example if original is in Downloads:

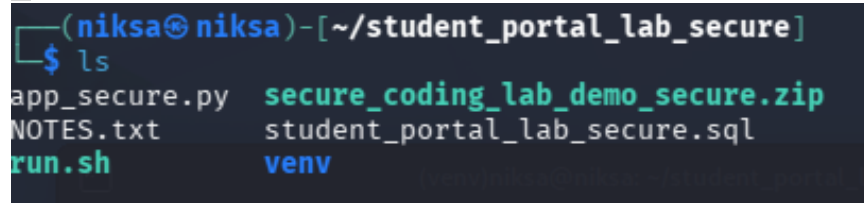
```
cp ~/Downloads/secure_coding_lab_demo_secure.zip ~/student_portal_lab_secure/
```

```
cd ~/student_portal_lab_secure
```

- Unzip it:

```
unzip secure_coding_lab_demo_secure.zip
```

```
ls
```



```
(niksa@niksa)-[~/student_portal_lab_secure]  
$ ls  
app_secure.py  secure_coding_lab_demo_secure.zip  
NOTES.txt     student_portal_lab_secure.sql  
run.sh        venv
```

Step 2 - Update Kali and install required system packages

These packages are needed for:

- python3 → run Flask app
- python3-pip → install Python libraries
- python3-venv → create isolated environment
- mariadb-server → database backend
- unzip → extract the package

- First update package list:

```
sudo apt update
```

- Install the required tools:

```
sudo apt install -y python3 python3-pip python3-venv mariadb-server unzip
```

Step 3 — Start and enable MariaDB

- Start the database service:

```
sudo systemctl start mariadb
```

```
sudo systemctl enable mariadb
```

Check status:

```
sudo systemctl status mariadb
```

It should be up and running

```
(niksa@niksa)-[~/student_portal_lab_secure]
$ sudo systemctl status mariadb
[sudo] password for niksa:
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; prese
   Active: active (running) since Mon 2026-04-06 08:55:53 EDT; 1h 13min ago
   Invocation: e390292fda1747cf9c9ab9cee486935b
   Docs: man:mariabdb(8)
```

Step 4 — Create the lab database user

- Open MariaDB as admin:

```
sudo mysql
```

- Run these SQL commands:

```
CREATE DATABASE IF NOT EXISTS student_portal_lab_secure;
CREATE USER IF NOT EXISTS 'labuser'@'localhost' IDENTIFIED BY 'labpass123';
GRANT ALL PRIVILEGES ON student_portal_lab_secure.* TO 'labuser'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

The SQL commands create:

- database: student_portal_lab_secure
- user: labuser
- password: labpass123

Step 5 — Import the application database

Next to imports:

- users
 - assignments
 - messages
 - sample accounts and sample records
- From your project folder:

```
mysql -u labuser -p student_portal_lab_secure < student_portal_lab_secure.sql
```
 - When prompted, enter the lab password (labpass123).

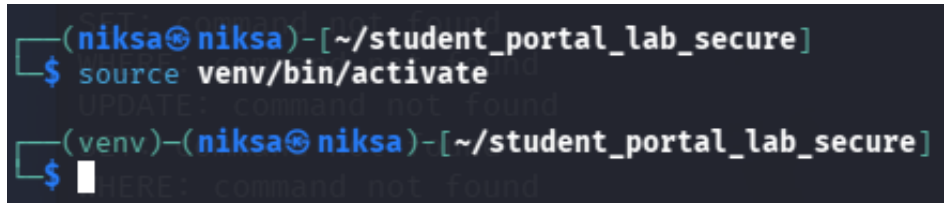
Step 6 — Create a Python virtual environment

Inside the project folder:

```
python3 -m venv venv  
source venv/bin/activate
```

- The command means:
 - **-m venv**: Tells Python to run the built-in venv module.
 - **venv**: The name of the folder where the environment will be stored. You can name this anything (e.g., .myenv), but venv is the standard convention.

You should now see (venv) in your terminal prompt.



```
(niksa@niksa)-[~/student_portal_lab_secure]  
$ source venv/bin/activate  
UPDATE: command not found  
(venv)-(niksa@niksa)-[~/student_portal_lab_secure]  
$
```

Step 7 — Install Python dependencies

With *venv* active, install the required libraries:

```
pip install flask pymysql werkzeug markupsafe
```

Step 8 — Configure environment variables

The secured app reads secrets and DB settings from environment variables.

Run:

```
export FLASK_SECRET_KEY='change-this-to-a-random-secret-key'
export DB_HOST='localhost'
export DB_USER='labuser'
export DB_PASSWORD='labpass123'
export DB_NAME='student_portal_lab_secure'
```

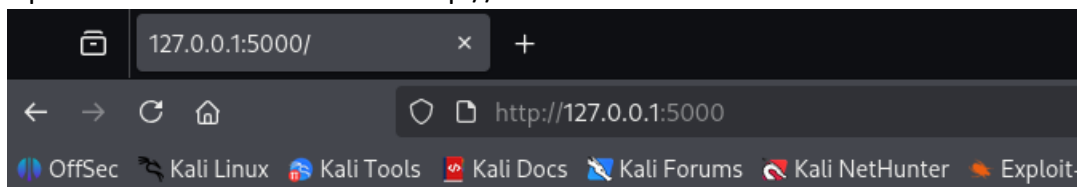
Step 9 — Run the secured application

```
python app_secure.py
```

The Flask app should start on: <http://127.0.0.1:5000>

```
(venv)-(niksa@niksa)-[~/student_portal_lab_secure]
$ python app_secure.py
* Serving Flask app 'app_secure'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [06/Apr/2026 09:28:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [06/Apr/2026 09:28:45] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [06/Apr/2026 09:28:52] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [06/Apr/2026 09:28:52] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [06/Apr/2026 09:29:05] "GET /assignment HTTP/1.1" 200 -
127.0.0.1 - - [06/Apr/2026 09:29:07] "GET / HTTP/1.1" 200 -
```

Open browser in Kali and visit: <http://127.0.0.1:5000>



Student Portal Lab Demo (Secured Reference)

[Login](#)

- [Assignment Submission](#)
- [Messaging](#)

Step 10 — Test login with sample accounts

Use these sample credentials:

- student1 / password123
- student2 / password123
- lecturer1 / password123
- admin1 / admin123

Troubleshoot (If login fail):

Run this exact command from Kali after activating venv:

```
python - <<'PY'
import pymysql
from werkzeug.security import generate_password_hash

conn = pymysql.connect(
    host="localhost",
    user="labuser",
    password="labpass123",
    database="student_portal_lab_secure",
    autocommit=True
)

users = {
    "student1": "password123",
    "student2": "password123",
    "lecturer1": "password123",
    "admin1": "admin123"
}

with conn.cursor() as cur:
    for username, password in users.items():
        cur.execute(
            "UPDATE users SET password_hash=%s WHERE username=%s",
            (generate_password_hash(password), username)
        )

print("Password hashes updated successfully.")
PY
```

After that, restart the app:

```
python app_secure.py
```